

1. The Core Strategy: Business-First Architecture

In any enterprise architecture, Disaster Recovery must be a business-first strategy, not just a technical one. We define our architecture based on two critical business metrics:

- **RPO (Recovery Point Objective):** This defines the maximum acceptable data loss measured in time. Essentially, it answers: "How many minutes or hours of data can the business afford to lose permanently?"
- **RTO (Recovery Time Objective):** This defines the maximum acceptable downtime. It answers: "How long can the business afford to have the system offline before it is fully restored?"

Our architecture directly maps to these requirements. For tier-1 applications with zero-RPO and near-zero RTO, we design **Multi-Region Architectures**. We choose between two primary models:

- **Active-Active Configuration:** Traffic is routed to both regions simultaneously. This offers the lowest RTO but comes at a significantly **higher cost**.
- **Active-Passive Configuration:** The secondary region remains on standby. If the primary region goes down, we initiate a failover. This option is highly **cost-effective** but introduces a slightly higher RTO.

2. Backup Strategies and Policies

To guarantee data resilience, we implement a strict backup hierarchy using native Azure services.

Azure Backup Infrastructure

We separate our backup management based on the resource type:

- **Recovery Services Vault:** Used for protecting heavy compute and infrastructure workloads, specifically **Azure VMs**, **Azure SQL databases**, and **Azure Files**.
- **Backup Vault:** Used for newer, cloud-native workloads like **Azure Disks** and **Azure Blobs (Storage Accounts)**.

Backup Lifecycle Policy

Our retention policy follows a standard grandfather-father-son compliance model:

- **Daily backups** for short-term recovery.
- **Monthly and Yearly backups** for long-term compliance, utilizing **incremental backups** to optimize storage costs.
- **Soft Delete:** Enabled across all vaults, storage accounts, and keys for **7 days** to protect against accidental deletion or ransomware attacks

• 3. Component-Specific DR & Backup Implementations

When designing the workload layer, we break down our disaster recovery mechanisms by component:

- **Compute (VMs):** We use **Azure Site Recovery (ASR)** to replicate VMs to a secondary global region for continuous data replication and low-RTO failovers. For standard backups, we use **Azure Backup**.
- **Databases (Azure SQL):** We leverage **Native Backups** combined with **Geo-Replication** to maintain a readable secondary database in our companion region.
- **Storage Accounts:** We enforce **Geo-Redundant Storage (GRS)** or **Read-Access Geo-Redundant Storage (RA-GRS)**. We also enable **Blob Versioning** and **Soft Delete** to protect historical state.
- **Azure Kubernetes Service (AKS):** We backup cluster configurations and persistent volumes using Velero or Azure-native AKS backup solutions, ensuring state can be rehydrated instantly in a secondary cluster.

• 4. Infrastructure as Code (IaC) & The Platform Layer

A modern DR strategy differentiates between **Workload Data** and the **Platform Layer**. While workload data relies on traditional replication and backups, our infrastructure does not.

Our **Networking, Landing Zones, Management Groups, Subscriptions, Azure Policies, RBAC, DNS, and Firewall Policies** are entirely managed as code using **Terraform or Bicep**.

- **Version Control & CI/CD:** Every piece of infrastructure is stored in Git, utilizing strict pull request approvals and secure state management.
- **The Immutable Architecture Principle:** If a regional disaster occurs, we do not attempt to patch the existing platform. Instead, we use our CI/CD pipelines to completely rebuild the entire landing zone and infrastructure stack from scratch in the secondary region. The infrastructure is treated as completely replaceable, allowing us to restore the entire network topology quickly and consistently.

• 5. Validation and Continuous Testing

An untested DR strategy is a failed DR strategy. To ensure operational readiness, we mandate a strict validation framework:

- **DR Drills:** We schedule and execute full DR drills **every 3 months** without disrupting production traffic.
- **Testing Scope:** These drills include comprehensive **Backup Restore Tests, Controlled Failover Tests** to the secondary region, and **Failback Validation** to ensure data integrity remains intact when returning to the primary region.
- **Documentation:** Following every drill, we perform a post-mortem review and immediately update our **Runbooks and DR documentation** to capture any changes in the environment layout